

EVMBT: A Binary Translation Scheme for Upgrading EVM Smart Contracts to WASM

Weimin Chen
The Hong Kong Polytechnic
University
Hong Kong SAR, China
cswchen@comp.polyu.edu.hk

Heming Cui
The University of Hong Kong
Hong Kong SAR, China
heming@cs.hku.hk

Xiapu Luo*
The Hong Kong Polytechnic
University
Hong Kong SAR, China
csxluo@comp.polyu.edu.hk

Shuyu Zheng
Peking University
Beijing, China
zhengshuyu@pku.edu.cn

Haoyu Wang*
Huazhong University of Science and
Technology
Wuhan, China
haoyuwang@hust.edu.cn

Xuanzhe Liu
Peking University
Beijing, China
liuxuanzhe@pku.edu.cn

Abstract

Ethereum is the first and largest blockchain that supports smart contracts. For better scalability and security, one major planned change of Ethereum 2.0 (Eth2) is to upgrade the smart contract interpreter from Ethereum Virtual Machine (EVM) to WebAssembly (WASM). In the meanwhile, many other popular blockchains have adopted WASM. Since Ethereum hosts millions of smart contracts, it is highly desirable to automatically migrate EVM smart contracts to WASM code to foster the prosperity of the blockchain ecosystem while inheriting the historical transactions from Ethereum. Unfortunately, it is non-trivial to achieve this purpose due to the challenges in converting the EVM bytecode of smart contracts to WASM bytecode and adapting the generated WASM bytecode to the underlying blockchain environment. In particular, none of the existing tools are adequate for this task because they fail to achieve accurate translation and compatibility with the blockchain environment. In this paper, we propose a novel solution and use Eth2 as the target blockchain to demonstrate its feasibility and performance because Eth2 is highly attractive to both industry and academia. Specifically, we develop EVMBT, a novel EVM2WASM bytecode translation framework that not only ensures the fidelity of translation but also supports plugins to improve smart contracts. Extensive experiments demonstrate that EVMBT can successfully

translate real-world smart contracts with high fidelity and low gas overhead.

CCS Concepts: • Software and its engineering → Translator writing systems and compiler generators.

Keywords: binary translator, smart contracts, wasm

ACM Reference Format:

Weimin Chen, Xiapu Luo, Haoyu Wang, Heming Cui, Shuyu Zheng, and Xuanzhe Liu. 2024. EVMBT: A Binary Translation Scheme for Upgrading EVM Smart Contracts to WASM. In *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '24)*, June 24, 2024, Copenhagen, Denmark. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3652032.3657570>

1 Introduction

Since the launch of Ethereum in 2014, millions of smart contracts have been deployed on the platform, including popular applications like DeFi, which has locked over 100 billion USD in total value [34]. Ethereum Virtual Machine (EVM) executes EVM bytecode of smart contracts. Ethereum plans to evolve into Ethereum 2.0 (Eth2 [21]) to address scalability and security limitations. This transition mainly involves replacing the EVM with WebAssembly (WASM) because WASM is a new interpreter running WASM bytecode [18], which has higher security and efficiency and supports more programming languages than EVM. In the meantime, many popular blockchains such as Polkadot [12] and NEAR [11] have adopted WASM for executing smart contracts.

To foster the prosperity of millions of smart-contract-based applications on Ethereum, it is highly desirable to automatically migrate them to WASM blockchains while inheriting historical transactions. To this end, NEAR and Polkadot compile an EVM interpreter to the WASM contract [19, 26], which executes Ethereum transactions on WASM. However, this introduces overhead as their smart contract execution requires both EVM and WASM. Binary translation is an expected solution to discard EVM, which generally involves two typical steps. First, converting EVM bytecode to WASM bytecode

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. LCTES '24, June 24, 2024, Copenhagen, Denmark

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0616-5/24/06

<https://doi.org/10.1145/3652032.3657570>

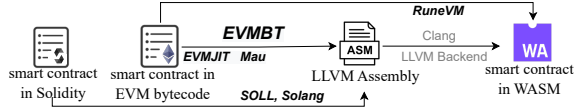


Figure 1. Tools for generating WASM smart contracts.

due to the limited availability of smart contracts source code (>99.9% are closed-source [3]) and compatibility across different programming languages [9, 14, 16, 17]. Second, enabling WASM contracts to interact with the underlying blockchain environment without relying on EVM. For instance, Eth2 integrates WASM with Ethereum Environment Interface (EEI [2]) and Ethereum Contract Interface (ECI [5]). EEI provides a blockchain environment for smart contracts, while ECI defines the smart contract bytecode structure necessary for code deployment in the blockchain.

Unfortunately, existing tools are inadequate for this task because they fail to achieve accurate translation and compatibility with the blockchain environment. In this paper, we propose EVMBT to address these challenges and use Eth2 as the target blockchain, which is attractive to both industry and academia [30, 51, 55, 58]. In § 6, we discuss how to extend EVMBT to other blockchains.

In particular, to automatically translate a smart contract, the translator needs to fulfill three requirements.

- **R1-Correctness:** it should generate WASM code with consistent behaviors as the corresponding EVM bytecode performs.
- **R2-Compatibility:** WASM contracts should support both ECI and EEI for the smart contract execution without EVM.
- **R3-Extensibility:** it should be extensible to support improvements to the created contracts.

Figure 1 depicts current efforts for WASM. Existing tools fail to meet **R1-R3** due to the limitations outlined in Table 1. Source code compilers (i.e., Solidity→WASM) must accommodate specific grammar rules that have evolved over past versions of Solidity, which can be error-prone and challenging (see § 5.1). For instance, Solang fails to recognize inline assembly and contract inheritance [8], undermining its effectiveness in generating accurate WASM code. Similarly, SOLL's immature support for ECI leads to the incorrect deployment of WASM contracts due to the missing deployment arguments. Template-based efforts like RuneVM [13] try to directly replace EVM opcodes to WASM code but fail to pass the test suites used in SOLL or Solang. Not even to mention other requirements. The JIT compiler EVMJIT [4] converts EVM bytecode to LLVM ASSEMBLY [46], an intermediate representation that can be converted to WASM code by clang [25]. However, EVMJIT uses an incomplete jump table and fails to fully resolve the jumps in EVM bytecode. The missing control flow in the JIT'ed code results in inconsistent behaviors. Although Mau [59] is also an LLVM-based translator, it cannot generate WASM code because it is designed for GPU binary. Additionally, both EVMJIT and Mau lack compatibility with Ethereum due to the absence of EEI and ECI support. The detailed differences are described in § 7.

Table 1. Comparison between existing tools and EVMBT. A usable tool should pass its own tests.

Tool	Input	Usable	Binary Trans	EEI	ECI
EVMBT	Bytecode	✓	✓	●	●
RuneVM[13]	Bytecode	✗	✓	●	○
EVMJIT[4]	Bytecode	✗	✗	○	○
Mau[59]	Bytecode	✓	✗	○	○
SOLL[57]	Solidity	✓	✗	●	●
Solang[41]	Solidity	✓	✗	●	●

● Fully support ● Partially support ○ Missing

In this paper, we propose and develop EVMBT, the first practical EVM2WASM binary translator that fulfills **R1-R3**. Our approach lifts the EVM bytecode of a smart contract to LLVM ASSEMBLY, which covers all semantic aspects defined in the EVM specification, and then leverages LLVM plugins [46] and clang to enhance and generate the WASM bytecode, respectively. This enables smart contracts to support ECI and EEI for successful deployment and on-chain execution.

It is non-trivial to develop EVMBT due to two challenges. **C1:** The indirect jumps in EVM bytecode prevent static analysis from recovering all control flow [39] because the jump destination is the top item in the current stack, which is determined until EVM actually executed it (i.e., runtime). The challenge lies in the path explosion problem, where the number of possible execution paths grows exponentially. Missing any branch leads to a correctness violation. **C2:** The architecture differences between WASM and EVM prevent WASM contracts from replicating EVM bytecode structure for code deployment. During deploying a contract (see § 2.2), EVM bytecode extracts arguments and the executable from its code segment [60]. However, since the code segment is not accessible to the WASM bytecode, an extended ECI is required for WASM to deploy smart contracts to Ethereum without relying on EVM.

To tackle **C1**, we resolve indirect jumps to direct jumps using symbolic execution whenever feasible. For the remaining jumps, we recover a complete jump table and then employ a devirtualization technique to emulate EVM jumps as table jumps (§ 3.1). This enables EVMBT to recover all expected control flow. To address **C2**, we rewrite the constructor code of smart contract to WASM code (§ 3.2). This moves the constructor code and runtime code to the data segment of WASM. The corresponding data loaded in EVM bytecode is redirected to WASM. It enables WASM contracts to deploy WASM code as its runtime code without utilizing the inaccessible segment.

We implemented EVMBT and released it at [Figshare](#). Apart from the standalone compiler version, we also integrate EVMBT with a WASM interpreter as a shared library which Eth2 nodes like *geth* [6] can load to replace EVM. On such an Eth2 testbed, we conduct extensive experiments to evaluate EVMBT in terms of effectiveness, correctness, and efficiency. The experimental results demonstrate that EVMBT can successfully translate EVM contracts to WASM contracts with consistent behaviors and low gas overhead.

In summary, this paper makes the following contributions.

- We design EVMBT, a novel EVM2WASM binary translation framework for translating EVM contracts to WASM contracts.
- EVMBT enables users to develop plugins to extend contracts in LLVM ASSEMBLY for various purposes. We equip EVMBT with the capability of optimization in LLVM ASSEMBLY and five plugins for fixing vulnerabilities in smart contracts.
- We conduct extensive experiments to evaluate EVMBT. The results show that the translation success rate of EVMBT reaches 100%, and the accuracy of execution is much higher than the baselines. EVMBT can generate gas-efficient contracts as source code compilers do (0.68% and 0.15% more gas than SOLL and Solang, respectively). EVMBT is also effective in fixing the vulnerabilities in SmartBugs dataset [37] with small code size overhead (3.4%) and gas overhead (9.3%).

2 Background

2.1 EVM Smart Contracts

The smart contract is typically written in Solidity [16] and then compiled into an EVM bytecode, which is a sequence of opcodes and will be deployed to the blockchain and executed by EVM. Each executed opcode costs some gas units as the execution fee [60]. EVM opcode is a stack-based language that uses a FIFO stack, linear memory, and key-value storage. Apart from stack opcodes (i.e., PUSH and POP), EVM can execute domain-specific opcodes to interact with blockchain environment, such as storage accesses (SLOAD, SSTORE), self code copy (CODECOPY), deposit amount records (CALLVALUE), cross-contracts calls (CALL), and log records (LOG).

2.2 Smart Contract Deployment with ECI

Ethereum Contract Interfaces (ECI [5]) defines that the bytecode of a smart contract must have exactly three parts: 1) constructor code: the function initiates and deploys the smart contract; 2) runtime code: the fragment of the constructor code that is returned and is set to be the bytecode of the smart contract that is callable by users after deployment; 3) the attached constructor arguments. When deploying a smart contract, EVM executes its constructor code with the constructor arguments and eventually obtains the runtime code. In addition, both the constructor code and its arguments will be discarded after the runtime code being deployed, making the runtime code become the new execution entry.

2.3 Ethereum Wasm

WebAssembly (WASM [18]) is the interpreter for WASM bytecode. Following ECI [5], WASM contract should initiate itself according to the arguments and upload another WASM executable to the interpreter as the runtime code for performing the deployment process mentioned in § 2.2. In addition, WASM contract needs Ethereum Environment Interface (EEI [2]). EEI defines a set of WASM methods with the same effects as EVM

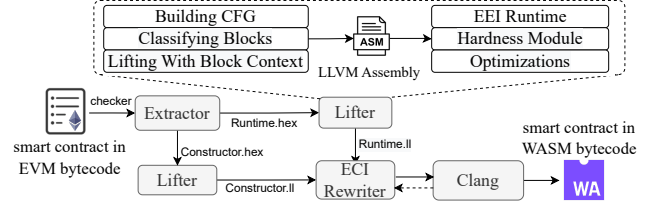


Figure 2. The architecture of EVMBT

domain-specific opcodes, enabling WASM contracts to interact with Ethereum without EVM.

3 EVMBT

Figure 2 depicts the architecture of EVMBT, which translates an EVM contract to a WASM contract via a six-step process pipeline. First, CHECKER verifies the format of the given EVM bytecode. If it follows the ECI structure, EXTRACTOR extracts the runtime code (Runtime.hex) and constructor code (Constructor.hex) from the bytecode, based on the opcode pattern (CODECOPY; *; RETURN; STOP). This pattern is learned from the format of transactions for contract deployment. Second, LIFTER lifts Runtime.hex and Constructor.hex to Runtime.ll and Constructor.ll in LLVM IR, respectively. More preciously, LIFTER builds a CFG of EVM bytecode to translate the stack-based opcodes to register-based IR. To approach C1, § 3.1.1 transforms EVM jumps to direct jumps with symbolic execution whenever feasible and lifts the remaining jumps to table jumps [46]. In the third step, EVMBT addresses EEI support. Fourth, in § 3.2, ECI-REWRITER enforces Constructor.ll to deploy the executable compiled from Runtime.ll (C2). Fifth, EVMBT utilizes IR-based plugins for performance optimizations (§ 4.1) and bug fixing (§ 4.2). Finally, clang generates a WASM contract from the LLVM ASSEMBLY.

3.1 LIFTER

The register-based LLVM IR is capable of covering all semantic aspects defined in EVM specification. Either EVM stack or memory is emulated by an LLVM array named as @mstk and @mem, respectively. Each element in @mstk has 32 bytes, being consistent with the size of EVM stack. @mem is a byte-addressable memory since the linear memory in EVM at least stores or reads one byte. Additionally, %sp is an LLVM register to maintain the current stack depth. Smart contracts can adjust %sp to obtain opcode operands and update the EVM stack. For instance, PUSH %data can be lifted to %sp = add %sp, 1; store %data, (getelementptr @mstk, 0, %sp). However, a full-emulation approach used in existing binary translators [29, 35, 59] will introduce too much overhead because all stack operations in EVM bytecode are lifted to the slower memory accesses. The key to efficient translation is determining the operands of opcodes at compile-time rather than deferring them all to the runtime as the emulation-based approach does. Fortunately, we observe that some opcodes may use the operands defined by the prior opcodes along

Algorithm 1 Lifting EVM bytecode to LLVM ASSEMBLY**Input:** bs : EVM basic blocks**Output:** LLVM ASSEMBLY

```

1:  $E, Q \leftarrow \emptyset, \langle bs_0, \emptyset \rangle$  // start from the entry basic block
2: while  $Q \neq \emptyset$  do
3:    $b, c \leftarrow Q.pop()$  // prioritize to successors
4:    $E.add(b)$  // mark the visited blocks
5:   compute  $\epsilon$  // see Algo 2
6:    $c' \leftarrow c \cup \{\text{@mstk}[\%sp-i] \mid (vis(b)?|c| : 0) \leq i < \epsilon\}$ 
7:    $\%sp \leftarrow \%sp - |c'|$ 
8:    $c' \leftarrow Lift(c', b)$  // lift opcodes
9:   emulate to push the results from  $c'$  to @mstk
10:   $Q \leftarrow Q \cup \{\langle j, c' \rangle \mid j \in succs[b] \setminus E\}$ 
11: end while

```

the path. Based on this observation, LIFTER designs an extended emulation-based approach, which infers operands from the CFG whenever feasible. This section will introduce this extended emulation-based approach.

3.1.1 Building EVM CFG. We identify EVM basic blocks based on Mau [59]. Additionally, we label each basic block with the PC (program counter) of its head opcode. Once basic blocks are established, LIFTER performs symbolic execution to build a control flow graph. The symbolic execution engine starts from the entry opcode with $PC = 0$ and simulates the execution. If the destination of an EVM jump is a constant, it further jumps to the corresponding opcode and continues symbolic execution until all resolvable control flow is identified. For example, the jump in “[PUSH EE; PUSH 1; ADD; JUMP]” is resolvable because we can infer that its destination is 0xEF and eventually lift to a direct jump.

Note that symbolic execution cannot resolve all EVM jumps due to the path explosion [1, 32, 38, 39]. For those remaining jumps that have implicit jump destinations, we resolve them until smart contract runtime (C1). Our solution is to recover all possible destinations of the indirect jumps into a jump table. EVM defines that any jumping destination must be a basic block starting with an opcode JUMPDEST. Thus, we can recover a **complete** jump table for any indirect jump that static tools cannot resolve. For correctness, we transform the EVM jumps to table jumps in LLVM ASSEMBLY, which indirectly jump to specific basic blocks until WASM runtime, according to the complete jump table recovered at compile-time.

3.1.2 Classifying Basic Blocks. LIFTER classifies basic blocks into two categories and applies different code generation strategies to each kind in § 3.1.3. The operands of the EVM opcodes used in one basic block may be defined in predecessors instead. The number of such operands is denoted as ϵ . For example, the ϵ of the basic block “[JUMP]” is one because JUMP requires the stack top value, which was not defined in this basic block but its predecessors. Thanks to the

CFG recovered from § 3.1.1, some basic blocks have explicit predecessors, which enables LIFTER to infer the operands at compile-time to reduce the uses of @mstk and improve efficiency. For a clear explanation, we define the *context* of an opcode as the collection of data existing in the stack right before its execution. We obtain the context of all opcodes in one basic block together, namely *block context*. The basic blocks are classified whether they are *context visible*. A basic block has visible context, meaning all operands used within it are defined by itself or its predecessors. According to the ϵ computed from Algorithm 2, we classify EVM basic blocks:

$$vis(block) = \begin{cases} true, & \text{fall-through} \vee (\text{jump-target} \wedge \epsilon = 0) \\ false, & \text{otherwise} \end{cases}$$

fall-through indicates the basic block is in the “else” branch of an if-then-else control flow. Its only predecessor is the “if” block, which can be exactly identified from the CFG. Thus EVMBT can infer the ϵ operands from the visible block context of the predecessor if ϵ operands are used in this basic block. Note that the basic block in the entry point of bytecode is also processed as a *fall-through* block. For the basic blocks starting with JUMPDEST, we name them *jump-target*. When $\epsilon = 0$, all operands used in this basic block have already been defined by the opcodes of this basic block. Thus, EVMBT can still infer the operands. Otherwise, LIFTER has to label the block context of *jump-target* blocks as invisible and apply the emulation-based approach (for example, @mstk[%sp]). That is because, as long as any indirect jump remains un-resolvable after § 3.1.1, all *jump-targets* blocks may become its destination, and their predecessors remain implicit. The un-resolvable control flow prevents EVMBT from inferring the operands used in one basic block but defined outside.

3.1.3 Lifting with Block Contexts. LIFTER chooses EVM basic blocks to lift by traversing the CFG in breadth-first search (BFS). Starting from the entry block, LIFTER uses a blank block context to lift the opcodes inside it. The stack results at the exit of one basic block become the block context of the successors. We describe the detailed procedure in Algorithm 1. In each BFS round, we handle one basic block and its block context, denoted as b and c (Line 3). c is a stack-structure vector maintaining the allocated LLVM registers of a block context. If b is context visible (see § 3.1.2), it means that ϵ operands used in b are defined in the context of b ’s predecessor. When the block context of b is invisible, LIFTER will create enough registers in LLVM memory (i.e., @mstk) to emulate these operands (Line 6). At Line 7, to balance stack, we decrease %sp by the length of c' . Once ϵ operands are identified in c' , we lift the opcodes one by one. At Line 8, following the lifting rules defined in Table 6, $Lift(c', b)$ creates LLVM ASSEMBLY under basic blocks. Check our implementation for the lifting rules for all kinds of EVM opcodes. The LLVM ASSEMBLY uses registers from c' as operands and defines new registers to compute its execution results. c' maintains the

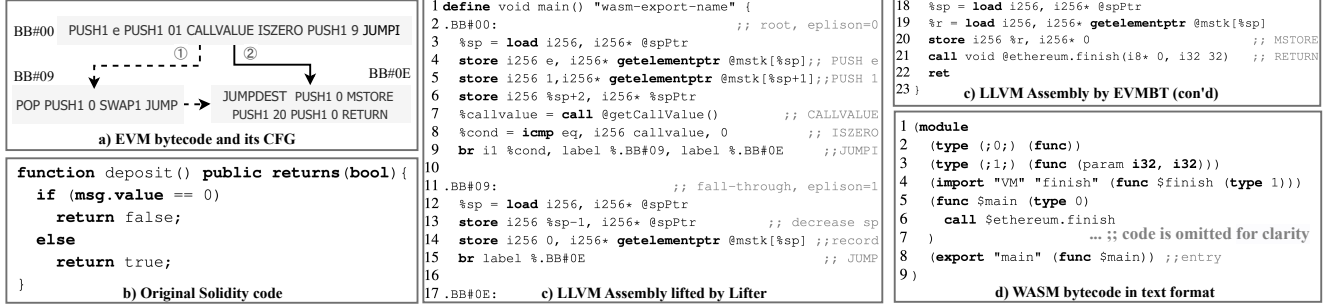


Figure 3. An LLVM ASSEMBLY example lifted from EVM bytecode. The assembly code was simplified for readability.

block context after the execution of the current basic block, i.e., b . To retain the stack results of b in LLVM memory, we further emulate to push the elements in c' in $@mstk$ at Line 9. At last, we extend the BFS queue and lift the successors of b , denoted as $succs[b]$.

Figure 3(a) shows the EVM bytecode compiled from the snippet code of Solidity listed in Figure 3(b). Figure 3(c) shows the lifted LLVM ASSEMBLY, which can be generated into the WASM code in Figure 3(d). As shown in Figure 3(a), two paths are identified in the EVM bytecode. If a caller invokes the smart contract with cryptocurrency ($CALLVALUE \neq 0$), the smart contract returns true directly. Otherwise, it returns the false result introduced from BB#09 along the path labeled in a dashed line. To lift the above example to the LLVM ASSEMBLY shown in Figure 3(c), EVMBT creates an LLVM function to represent the entire runtime code of the EVM bytecode. It handles BB#00 first, where the used operands are all defined inside itself. EVMBT creates $%callvalue$ to lift the CALLVALUE at Line 7. Next, EVMBT lifts a successor, i.e., BB#09, whose ϵ is one. Based on the block context of BB#00, we assign the operand of JUMP to 0x0e and settle down a direct jump to BB#0E. According to § 3.1.2, BB#0E has invisible context, thus $%r$ is defined to fetch the stack top from $@mstk$ (Line 19). At last, $%r$ is stored in memory as the return value used at Line 21. `getCallValue()` and `finish()` are two EEI methods for lifting CALLVALUE and RETURN, respectively.

3.2 ECI-REWRITER

A WASM smart contract should expose ECI to WASM environment for a successful deployment and invocation. To this end, ECI-REWRITER makes sure the Constructor.ll should not only extract constructor arguments from the EVM bytecode but also deploy a WASM executable as the runtime code. Without ECI support, the created smart contracts are incompatible with the existing Web3 infrastructures. The architecture differences between WASM and EVM prevent WASM contracts from replicating EVM contract structure to deploy itself. On the one hand, WASM does not allow extracting data from its code segment as EVM does for loading constructor arguments. On the other hand, the constructor code intends to return an EVM bytecode rather than WASM code.

To enable WASM smart contracts to initiate themselves according to the constructor arguments during the contract deployment, we allocate an LLVM string named $@evmCode$ with the value as the three parts of EVM bytecode. Since CODECOPY in EVM bytecode is the only opcode that can load the constructor arguments from its code segment, we transform it as a memory copying. According to the position information provided in EVM operands of CODECOPY, EVMBT creates LLVM ASSEMBLY to slice the content of $@evmCode$ to get the constructor arguments used in EVM bytecode. We use an example shown in Figure 4 to explain how to rewrite Constructor.ll. $@evmCode$ is a buffer which maintains the original EVM bytecode. At Line 4, it copies 32 bytes from the end of $@evmCode$ as constructor arguments.

To enforce WASM smart contracts to deploy WASM code, we rewrite the exit of Constructor.ll. We remove the EVM runtime code and make it return another piece of LLVM memory that maintains the lifted WASM runtime code. To be specific, we allocate an LLVM buffer named $@rtCode$ and initiate it as the WASM runtime code translated from Runtime.ll. The terminator, or the exit instruction of the rewritten Constructor.ll, would be an EEI call, where a `finish()` takes $@rtCode$ and deploys it. Figure 5 shows a classic EVM constructor and the LLVM ASSEMBLY that we generate. $%ptr$ is the pointer of $@rtCode$. At Line 3, the `finish()` loads data from $%ptr$ to $%ptr+1024$ and returns a bytecode to WASM for deploying runtime code.

4 Extensible Plugins

EVMBT provides a uniform platform for developers designing optimizations and hardness. Each plugin is an LLVM Pass, which accepts an LLVM ASSEMBLY as the input and outputs the enhanced LLVM ASSEMBLY.

4.1 Optimization Module

LLVM Default Optimizations. EVMBT enables various optimizations from LLVM framework.

Function Signature. Upon execution entry, the smart contract in EVM reads the first four bytes of user input as the function signature to determine which function is invoked. Since the word width of EVM is 32 bytes, EVM smart contracts have to load the input as an `uint256_t` variable first and divide

```

PUSH1 20 PUSH2 01e0 PUSH1 00 CODECOPY
1 @evmCode = constant 0x608060...
2 %dst = inttoptr i256 0 to i8*
3 %src = getelementptr @evmCode, i32 0, i32 480
4 call void @memcpy(i8* %dst, i8* %src, i32 32)
    
```

Figure 4. Loading the uint256_t constructor argument.

```

PUSH2 0160 DUP1 PUSH1 80 PUSH1 00 CODECOPY PUSH1 00 RETURN
1 @rtCode = constant c"\00asm\01\00...";;1024B
2 %ptr = i8* getelementptr @rtCode, i32 0, i32 0
3 call void @finish(%ptr, i32 1024)
    
```

Figure 5. Returning the runtime code.

it with 2^{24} to get the function signature. In LLVM, we directly obtain the expected value by directly loading a uint32_t data from the user input.

256bits Arithmetic Operations. In EVM, arithmetic operations are all 256-bit type., while native WASM data is limited to 64 bits. To optimize arithmetic operations, we utilize four 64-bit registers as one 256-bit data [45].

Framework. We mainly explain how a plugin should be designed and adapted. Figure 6 shows the workflow of the hardness module, including *ExecDetector* and *Plugins*. When lifting EVM bytecode, the program counter (PC) of each EVM opcode is attached to the corresponding LLVM ASSEMBLY lifted, regarding as metadata. During the hardness procedure, *ExecDetector* collects the PCs of the vulnerable opcodes that are identified by existing detection tools. Next, according to the received PC metadata, *Plugins* locates vulnerable LLVM ASSEMBLY and applies patches. Developers can apply their detectors in *ExecDetector* and extend *Plugins* with LLVM Pass. As a demonstration, we employ existing tools [10, 31, 49] to analyze bytecode and design five plugins to fix the commonly-seen vulnerabilities using the oracles from existing auto-fixing studies [42, 50, 53, 61]. Designing a hardness tool that can outperform all prior tools is out of scope. We do not claim the completeness of the hardness but the extensibility of EVMBT that developers can design security plugins flexibly under the uniform framework.

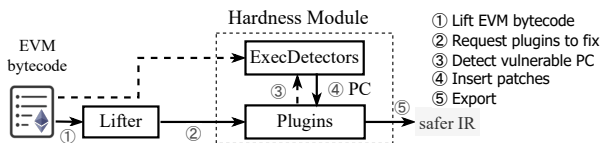


Figure 6. Hardness module for fixing vulnerabilities.

4.2 Hardness Module

Security Plugins. Due to space, we skip the vulnerability details and refer the readers to Appendix A.

• **P1: Integer Bug (IB).** We apply LLVM sanitizers to check arithmetic operations. A conditional jump terminates execution whenever any arithmetic overflow/underflow occurs.

• **P2: Intra-function Reentrancy (iRE)** iRE attackers take over the victim’s control flow by 1) invoking the victim and 2) enforcing it to callback itself within the same transaction. To patch it, we allocate a global variable as the mutex, which locks the smart contract states before the external call and unlocks it eventually. A conditional jump is inserted into the smart contract entry, which jumps to a termination when loading a locked mutex.

• **P3: Cross-function Reentrancy (cRE).** We lock every function with a mutex. It uses the same patch in P2.

• **P4: Dangerous tx.origin (OR).** We replace the vulnerable *tx.origin* with a safe call, i.e., *msg.sender*.

• **P5: Unchecked Send (US).** We insert an additional branch to check the result of the cross-contract call. It terminates the transaction if the return value is zero.

5 Implementation and Evaluation

Implementation. We implement EVMBT based on LLVM using approximately 8,000 lines of C++.

Eth2 testbed. We constructed a local blockchain (a.k.a. test-net) using a *geth-0x6c61eba* node [6], which incorporates *Hera-0xa396507* [7] as the WASM environment. This customized *geth*, equipped with dual interpreters, is able to run both EVM bytecode and WASM bytecode through unified interfaces. Although *Hera* is the state-of-the-art WASM runtime, it does not support all EEI methods. Improving *Hera* to be available in the industry deserves another paper. For experimental testing, we configured *Hera* to return pre-defined constants when responding to specific EEI APIs introduced from the Ethereum “London” upgrade.

Baselines and Setting. Since EVMBT is the first practical translator, we compare it with Solidity→WASM compilers such as SOLL and Solang. We run them with the default setting declared at their GitHub repositories. Although both SOLL and Solang have announced to stop supporting WASM, they are still the best baselines for Eth2 as far as we know. RuneVM is excluded because it fails to compile any test suites used in SOLL and Solang. All experiments were run on a server running Ubuntu 18.04 with an Intel i9 CPU and 64 GB RAM.

Benchmarks. We construct three benchmarks for evaluating EVMBT such as **B1**, **B2**, and **B3**.

► **B1.** We randomly collected 2,000 real-world smart contracts written in Solidity from Etherscan [3], along with their on-chain transactions. As shown in Figure 8, the average lines of code (LOC) of each contract is 741. Contracts were written using various versions of Solidity, ranging from v0.4 to v0.8. We generate 1,983 EVM bytecode according to their compiling parameters, which is available in Etherscan. The average size of EVM bytecode is 25.5 KB, and the average compiling time is about 40.0 ms. Not all source files could be successfully compiled into EVM bytecode because the compiling config downloaded from Etherscan may be insufficient. Note that

Table 2. The effectiveness of the different tools on B1.

Bytecode Size	[0,10KB)	[10,20KB)	[20,30KB)	[30,40KB)	[40,Max)
EVMBT	419 (100%)	549 (100%)	384 (100%)	243 (100%)	387 (100%)
SOLL	14 (3.34%)	3 (0.55%)	1 (0.26%)	0 (0.00%)	0 (0.00%)
Solang	83 (19.81%)	29 (5.28%)	2 (0.52%)	3 (1.23%)	0 (0.00%)
Total	419	549	384	243	387

EVMBT does not need source files. We collect open-source smart contracts to run the baselines.

► **B2.** Based on the experimental results on RQ1, we identify serious usability issues with the existing tools when compiling real-world smart contracts. To address this, we have curated a synthesis benchmark, called **B2**, to ensure all baselines can achieve 100% transaction correctness. **B2** comprises fourteen smart contracts sourced from public test suites (see Figure 7), including the Ethereum official tests [22]. Each contract repeats the corresponding computation 10,000 times. We listed the details of the benchmark design in the [supplemental material](#).

► **B3.** To evaluate the effectiveness of the fixing, we create **B3**, which includes five types of vulnerabilities mentioned in § 4.2. This benchmark consists of 36 EVM smart contracts sourced from SmartBugs [36, 37], a reliable dataset for evaluating existing bug detectors. In addition to the smart contracts, we also constructed 204 transactions that cover all branches within these smart contracts.

Research Questions. Our evaluations are driven by five research questions: (i) **RQ1:** Can EVMBT successfully translate EVM bytecode into WASM bytecode?, (ii) **RQ2:** Can EVMBT achieve the semantic correctness?, (iii) **RQ3:** Is the overhead introduced by EVMBT acceptable?, (iv) **RQ4:** Can the context recovery and optimization strategies reduce redundant code?, and (v) **RQ5:** How is the performance of the plugins (**P1-P5**) in fixing vulnerabilities?

5.1 RQ1: Effectiveness

Approach. We compare EVMBT and the two baseline approaches over **B1**. A translation is successful if the WASM bytecode is generated without raising any error.

Results. Out of 1,983 EVM bytecode from **B1**, EVMBT achieves a 100.0% success rate. Table 2 presents the success rates of different tools for translating EVM smart contracts of various sizes, with the maximum size of EVM bytecode being 111KB. On the other hand, when evaluating SOLL and Solang using Solidity source code, the results are quite surprising. They only handle a small fraction of the samples, with success rates of 0.9% and 5.9%, respectively. This is primarily due to their failure to recognize certain grammar patterns like inline assembly and contract inheritance. As a result, SOLL and Solang can only handle simpler examples. For instance, the average Solidity size of the successfully translated examples in SOLL is 58 LOC, while for Solang, it is 142 LOC. By contrast, the average size of **B1** is 741 LOC.

Table 3. B1 transaction effects.

Tool	Contracts #1,162	Txs #6,854	Same Txs	Score
EVMBT	1,162 (100%)	6,854 (100%)	6,798 (99.2%)	99.998%
SOLL	7 (0.60%)	17 (0.24%)	0 (-)	80.6%
Solang	23 (1.97%)	5 (0.07%)	0 (-)	87.8%

Answer to RQ1: EVMBT reaches a high success rate in binary translation. It is even more effective than SOLL and Solang to generate WASM contracts.

5.2 RQ2: Correctness

Approach. Correctness is a crucial metric for a binary translator. To determine that the generated WASM contracts remain consistent with the original EVM contracts, we employ a differential testing that involves deploying smart contract in **B1**, replaying transactions on the testnet, comparing the execution traces between EVM and WASM, and calculating a similarity score, denoted as $\frac{2 * M(t_1, t_2)}{|t_1| + |t_2|}$. $M(t_1, t_2)$ is the number of gestalt pattern matches [56] between the two traces, e.g., t_1 and t_2 . $|t|$ is the token amount of t .

► **transaction collection.** 1,162 smart contracts and their 6,854 transactions in **B1** are used for the differential testing. The remaining contracts are removed for a fair comparison. We exclude 819 contracts that EVM fails to deploy due to the differences between the testnet and the real-world blockchain. Additionally, two EVM contracts need the EEI method EXTCODEHASH, which Hera has not yet supported, which means their transactions cannot be replayed for correctness measurement. After excluding these problematic contracts, we proceeded with the remaining 1,162 EVM contracts for further analysis. During the transaction replaying process, we executed the transactions on the testnet and excluded transactions that yielded different results twice on EVM. Specifically, the excluded transactions have different outputs when EVM executed them twice. That is because the smart contracts are being executed in a different blockchain environment, specifically regarding the timestamp and block height. As a result, a total of 6,854 transactions were replayed to evaluate the correctness of EVMBT.

► **trace collection.** To capture the execution traces, we instrument EVM and WASM interpreter, respectively. Apart from the execution status and response of the replayed transaction, traces also contain the input and output of the operations (i.e., EEI methods) to Ethereum, such as storage accesses, events records, and cross-contract calls. If two transactions have the same EEI calls in terms of input, output, and executed order, we consider they have consistent effects regarding the smart contracts having consistent semantics. *To the best of our knowledge, this is the strictest verification, comparing the existing work, i.e., sGUARD [53], which only considers the execution status of transactions.*

Table 4. The runtime overhead on B1 of the WASM compared with the native EVM bytecode.

Tool	#Txs	Gas/unit	Gas Overhead	Time/ms	Time Overhead
EVM	6,854	76,748	0%	2.32	0%
EVMBT	6,854	77,370	0.81%	313	13380.67%
SOLL	17	34,641	2.34%	4	70.15%
Solang	5	48,129	-2.40%	5	127.13%

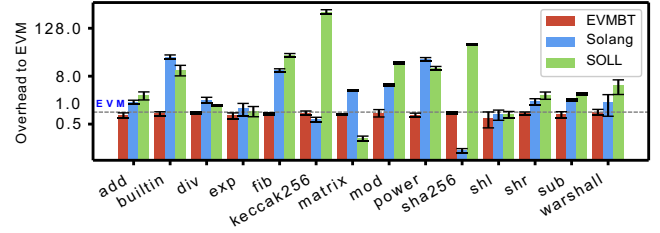
Results. A total of 1,162 WASM (100%) contracts translated by EVMBT were successfully deployed on the testnet. They have consistent behaviors with EVM smart contracts in terms of the similarity score of transaction traces, which outperformed the baselines in terms of correctness. By contrast, the two baselines only generated thirty deployable WASM smart contracts in total. The WASM contracts generated by EVMBT were able to execute 6,854 (100%) transactions, surpassing the contracts from SOLL (#17) and Solang (#5). The last column of Table 3 shows the similarity score between EVM transactions and WASM transactions. Among the executed transactions, EVMBT achieved a score of 99.998%, while SOLL and Solang scored 80.6% and 87.8%, respectively.

The 0.002% score loss of EVMBT is due to the immature WASM runtime rather than the code generation issues. Although 56 groups of transactions have inconsistent traces originating from 15 smart contracts, the remaining #6,798 (99.2%) groups of transactions produced exactly the same traces. Upon investigation, we discovered that the inconsistent traces are caused by the implementation differences between WASM and EVM interpreter. For instance, the smart contract *BDAONetwork* stores the current time in persistent storage, but the timestamp from the WASM runtime differs from the EVM interpreter in Hera, making WASM store inconsistent data to the storage. We further compared each inconsistent transaction trace with the reference EVM one and found that WASM and EVM invoked the same EEI methods as the EVM one, both in terms of function name and invoke order. This suggests that these transactions produce the same blockchain effects if the blockchain is resynchronized in the same environment. Despite these inconsistencies due to EEI, EVMBT still surpasses the source code compilers (SOLL and Solang) in terms of correctness.

Answer to RQ2: EVMBT can achieve high semantic correctness in translating EVM bytecode.

5.3 RQ3: Overhead

Approach. We measure the overhead in terms of the gas fee and execution time of the transactions. The gas fee is a direct measure of the computational effort required to execute operations in the blockchain. Transaction senders need to pay cryptocurrency to miners as the execution fee. Since baselines were unable to generate correct WASM code from B1 (see RQ2), we further use B2 to investigate the

**Figure 7.** Time overhead on B2.

overhead. The synthesis contracts on B2 ensure all tools reach 100% transaction correctness.

Results on Gas Overhead. Table 4 shows the overhead of the replayed transactions from B1, comparing with the native EVM bytecode. EVMBT introduces a gas overhead of only 0.81% to the translated WASM code. This indicates that EVMBT is able to generate efficient WASM code with minimal additional gas consumption. Since the baselines are ineffective in B1, we further measured the overhead using smart contracts that are available for both EVMBT and the baselines. In this comparison, EVMBT demonstrates similar gas efficiency as SOLL (+0.68%) and Solang (+0.15%).

Results on Time Overhead. Although smart contracts generated by EVMBT take 136x more times than EVM to execute the light-workload transactions in B1, EVMBT achieves 1.05-1.20x speed up for the B2 smart contracts which are mainly used for numeric computation. Overall, the overhead of EVMBT is acceptable.

► **B1.** Figure 9 illustrates the absolute overhead of the transactions that were executed on the WASM contracts generated by EVMBT. 93.0% of transactions were completed in less than one second in WASM, with the maximum delay being 2.7 seconds compared to the EVM contracts. Table 4 shows the relative overhead. On average, a transaction can be executed within 2.32 ms in EVM, but WASM requires 313 ms, making it 136 times slower. This large overhead is caused by the immature programming practices used in existing WASM runtime, particularly when executing cross-contract calls. Additionally, the WASM interpreter is not a native component of *geth*, which leads to a longer cold boot time compared to the EVM [63]. For instance, the source code compilers can only generate less-efficient WASM code as well. For instance, the WASM smart contracts generated by SOLL and SOLL run 1.7x and 2.27x slower than EVM code.

► **B2.** To investigate the overhead introduced by the binary translation itself, we conducted additional experiments on B2. The experimental setting remains unchanged (see Figure 7). On average, a transaction executed in EVM needs 62.9 ms, which is 27.4 times larger than the B1. More time portion is spent on the smart contract execution rather than the WASM cold boot. Surprisingly, except for the *warshall* item, all WASM code generated by EVMBT can execute faster than the corresponding EVM code, achieving a speedup ranging from 1.05x to 1.20x. The *warshall* smart contract, which runs the Floyd-Warshall algorithm, exhibited a 30% overhead when its EVM

Table 5. The size distribution of WASM code from B1.

Bytecode Size	[0,10KB)	[10,20KB)	[20,30KB)	[30,40KB)	[40,Max)
Native	5.65	14.57	26.07	34.90	55.83
SOLL	8.5 (2.1x)	0.3 (0.01x)	0.3 (0.01x)	N/A	N/A
Solang	6.5 (1.6x)	21.1 (1.5x)	26.2 (1.0x)	59.0 (1.8x)	N/A
EVMBT	55.0 (9.7x)	139.5 (9.6x)	266.2 (10.2x)	361.9 (10.4x)	599.6 (10.7x)
EVMBT ^{ctx}	142.0 (25.1x)	354.7 (24.3x)	631.4 (24.2x)	874.1 (25.0x)	1402.1 (25.1x)
EVMBT ^{opt}	55.5 (9.8x)	140.3 (9.6x)	267.2 (10.3x)	364.1 (10.4x)	602.3 (10.8x)

code was translated to WASM by EVMBT. However, EVMBT introduced less overhead to the WASM code compared to the baselines, even though EVMBT only takes EVM bytecode as input, whereas the baselines use source code. By contrast, SOLL and Solang introduced 1672.1% and 222.7% overhead on B2, respectively. On B2, EVMBT is more efficient because it focuses on general computation tasks, excluding heavy cross-contract calls. We argue that general computation in WASM is efficient, especially for heavy tasks. We believe that WASM smart contracts have the potential to achieve better performance in general computation tasks.

Answer to RQ3: EVMBT introduces an acceptable overhead due to the binary translation.

5.4 RQ4: Code Optimization

Approach. To evaluate the efficiency of block contexts and optimizations, we disable one of them (i.e., EVMBT^{ctx} and EVMBT^{opt}) to re-translate the smart contracts in B1.

Results. Table 5 shows the size distribution of the generated WASM, with the second row indicating the average size of the native EVM bytecode. The average size of WASM code from EVMBT is approximately ten times larger than the native EVM bytecode. For those samples available for all baselines, EVMBT generates 5.7x and 5.9x larger code than SOLL and Solang, respectively. It is worth noting that binary translation naturally results in larger code due to the absence of source code to identify high-level semantics. This trade-off between correctness and overhead is addressed in § 5.2-5.3, where we demonstrate that the trade-off is justifiable for high correctness. While SOLL and Solang generate slimmer code, they cannot compile Solidity and generate ground truth.

Nevertheless, we apply optimization techniques to reduce code size. Without enabling block contexts, the size of the WASM code reaches 631 KB, which is 2.4x larger than the equipped EVMBT. The optimization strategies employed reduce an average of 1,350 bytes (-0.51%). Additionally, we also measure the translation time. The block context enables EVMBT to generate slimmer LLVM ASSEMBLY, which accelerates the code generation. EVMBT^{ctx} needs about 18,598 ms for translating one sample, which is 4.7x longer than the equipped one. On average, the optimizations only require an additional 17 ms.

Answer to RQ4: Both block contexts and optimizations can reduce the size of the WASM contracts.

5.5 RQ5: Effectiveness of P1-P5

Approach. To evaluate the effectiveness of our six plugins for fixing vulnerabilities (§ 4.2), we utilize B3 as a ground truth dataset. For instance, we launch a reentrancy attack to drain the WASM victim’s funds and subsequently examine whether the applied patches can revert the attacker transactions and assess their impact on other benign transactions. Additionally, we analyze the overhead of one plugin by disabling the fixes and replaying the transactions, enabling a comparison of gas consumption between vulnerable and fixed contracts.

Results. The plugins are effective in terminating all 36 exploit transactions. The patches of IB bugs introduced 1.48 KB on average for each WASM smart contract. That is primarily due to EVMBT employing a coarse-grained strategy to locate vulnerabilities, which may result in redundant protections being inserted. For other vulnerabilities, the size of the patched contracts increases by an average of 520B (iRE), 570B (cRE), -4B (OR), and 0B (US). Among the plugins, P1 introduces the most code because it utilizes multiple sanitizers to verify arithmetic operations. Conversely, P4 may make the patched contracts smaller than the buggy ones. This is because the optimizations can remove redundant function headers if all tx.origin values are labeled as vulnerable and subsequently removed. The smart contracts patched by P5 remain the same code size and gas consumption, as P5 simply replaces one branch destination in the WASM code. Regarding gas overhead, each transaction requires an average of an additional 4,466 gas units (9.3%). This overhead is primarily introduced by the reentrancy fixes, where additional storage variables are needed, increasing storage access costs. For other types of vulnerabilities, the patches introduce negligible gas consumption, typically less than 1%. Full performance details are shown in the [supplemental material](#).

Answer to RQ5: EVMBT can successfully fix five kinds of vulnerabilities with low overhead.

6 Discussion

Migrate to Other Blockchains. We use Eth2’s SDK to demonstrate EVMBT’s capability of translating real-world smart contracts. We can migrate EVMBT to other blockchains. EVMBT contains an EVM2LLVM compiler, which lifts EVM bytecode into LLVM ASSEMBLY, enabling developers to translate Ethereum smart contracts to various binary code, such as native code for the just-in-time compilation, WASM for Polkadot [12] and NEAR [11], and BPF for Solana [15]. The minor efforts include 1) employing their SDK to replace EEI and ECI for interacting with the blockchain environment and code deployment and 2) enabling an LLVM backend to generate target code.

Translation. EVMBT has several limitations. First, for those smart contracts loading EVM opcodes from users to execute

dynamically, EVMBT cannot translate them directly because it is hard to recognize the dynamic loading at the static phase. One possible solution is using JIT technology, which dynamically translates the EVM bytecode.

Overhead. Second, EVMBT focuses on correctness rather than efficiency. Apart from collecting more smart contracts to evaluate EVMBT, we should improve RQ2 to collect the traces by re-syncing the entire blockchain rather than just executing transactions again. Besides, EVMBT extends the emulation-based approach to optimize IR and generates 2.4x smaller code. We will explore how to further improve its performance in future work since the 136x performance loss on **B1** is large. Nevertheless, WASM can speed up the complex smart contract execution on **B2** when the workload is heavy. To make it practical to use EVMBT in the production environment, it is necessary to support more optimizations.

Security Issues. Third, we reuse the existing vulnerability detectors and oracles in patching plugins. In the next stage, we will explore the design of detection methods based on LLVM ASSEMBLY rather than EVM bytecode. Although WASM provides a stackoverflow-free, sandboxed execution environment, its linear memory has weak security property [40, 47]. We bring control flow information into the memory for resolving indirect branches. It is possible for attackers to rewrite the jump destination for launching control hijacking attacks. We will examine how to further increase the security of WASM contracts automatically in future work.

7 Related Work

Binary Translation. The binary translation aims to migrate applications from one ISA (Instruction Set Architecture) to another one while remaining functionality. Researchers designed some translators that translate native binaries to WASM through LLVM IR [29, 33, 35, 52, 54]. However, they will introduce redundant IR because all x86 instructions and components are lifted to the slower memory operations [48].

Although Mau [59] is also an EVM translator, emulating everything (e.g., PC, memory, stack, and storage) in LLVM memory, there are major differences between EVMBT and Mau: 1) EVMBT generates WASM code while Mau is designed for GPU code generation; 2) EVMBT equips EEI, which Mau lacks; 3) EVMBT enables smart contract deployment with ECI, which Mau cannot support. Note that in addition to the architectural differences between GPU code and WASM, another factor that renders Mau infeasible for WASM is its inability to guarantee correctness. Since GPUs do not provide a native blockchain environment, Mau must simulate the EEI and ECI methods on the GPU, which leads to issues with correctness. Furthermore, Mau needs to emulate every EVM stack operand in LLVM IR, which introduces unnecessary redundancy. Although EVMBT shares similar ideas with Mau in terms of emulation-based translation, EVMBT introduces an extended approach that infers the operands of EVM

opcodes at compile time based on their use-define chain recovered from the control flow graph (CFG), which reduces redundant LLVM IR. Moreover, EVMBT enables EEI and ECI for blockchain interaction.

JVM to Wasm. Both EVM and JVM bytecode are stack-based machine code. Some tools target translating JVM bytecode to WASM [20, 24, 28]. These tools emulate the bytecode execution to identify the runtime variables and generate an SSA representation for WASM code generation. In particular, EVMBT has to resolve indirect jumps. JVM translators have no such challenge because JVM does not allow indirect jumps [23].

IR-based Analysis. ZEUS [44] is the first Solidity to LLVM IR tool to detect vulnerabilities in smart contracts. EXGEN [43] compiles Solidity into LLVM IR and then analyzes vulnerabilities above the SSA CFG. However, the LLVM ASSEMBLY that they generate is inconsistent with the source code in terms of semantics because they ignored some details for the convenience of static analysis. For example, EXGEN uses pseudo values to lift EEI opcodes, e.g., BALANCE. Even for those WASM compilers such as SOLL [57] and Solang [41], they cannot support certain grammar features in Solidity.

Smart Contract Hardness. Researchers have proposed several tools [42, 50, 53, 61, 62] to fix vulnerabilities in a smart contract before it is deployed to Ethereum. Existing tools fix vulnerabilities via rewriting EVM bytecode or Solidity, thus all of them are code-dependent approaches. In this paper, we proposed the first code-independent fixing framework. Based on LLVM, EVMBT not only benefits from various fixes in LLVM ecosystem but allows developers to update patterns for new patches via LLVM APIs.

8 Conclusion

Ethereum plans to upgrade EVM to WASM in Eth2. This move aligns with other WASM blockchain platforms. Translating EVM smart contracts to WASM is challenging, but the proposed tool, EVMBT, overcomes these challenges by ensuring accurate translation and compatibility with WASM. In addition, EVMBT supports LLVM plugins for enhanced security and performance optimization. Extensive experiments demonstrate that EVMBT successfully translates real-world EVM bytecode with high fidelity and generates gas-efficient contracts. With minor efforts on adapting blockchain-specific interfaces, EVMBT is extensible to apply to other WASM blockchains.

Acknowledgments

We thank the anonymous reviewers and shepherds for their valuable comments and instructions. This work is partly supported by Hong Kong RGC Projects (PolyU15224121, PolyU15231223), the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), the National Key R&D Program of China (2021YFB2701000), and the National Natural Science Foundation of China under the grant numbers 62325201.

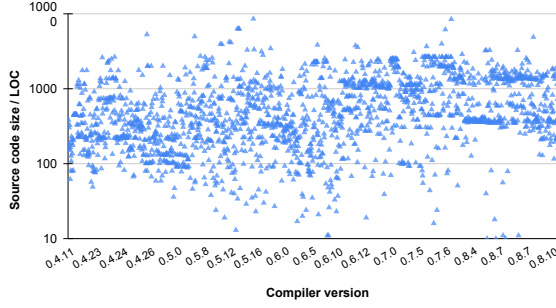


Figure 8. Profiles of the smart contracts in B1.

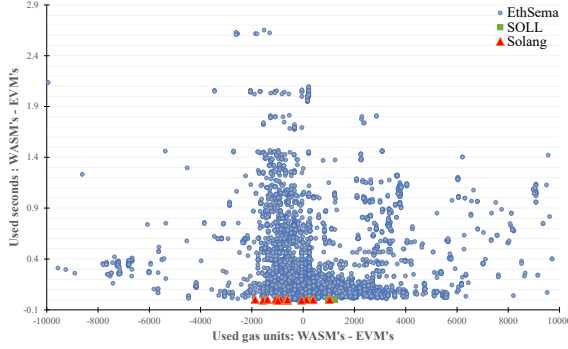


Figure 9. EVMBT's overhead on B1

Algorithm 2 Computing ϵ for one EVM basic block

Input: ops : the EVM opcodes of the given basic block

Output: ϵ

```

1:  $\epsilon, p \leftarrow 0$ 
2: for each  $\mu, v \in ops$  do  $\triangleright$  pops  $v$  values and pushes  $\mu$  data
3:    $\epsilon \leftarrow \epsilon + \max(0, v - p)$ 
4:    $p \leftarrow \max(0, p - v) + \mu$ 
5: end for
    
```

Table 6. Lifting rules. μ , l and p refer to `@mstk`, `@mem`, and `%sp`, respectively. Domain-specific opcodes are abstract in `eei()`. `table` is the jump table recovered in § 3.1.1.

op	IR	op	IR
push v	$p \leftarrow p + 1; \mu[p] \leftarrow v;$	mload	$i \leftarrow \mu[p]; \mu[p] \leftarrow l[i]$
pop	$p \leftarrow p - 1$	mstore	$v, i \leftarrow \mu[p - 1 :];$ $l[i] \leftarrow v; p \leftarrow p - 2$
unary	$v_1 \leftarrow \mu[p]; \mu[p] \leftarrow op\ v_1$	sload	$i \leftarrow \mu[p]; \mu[p] \leftarrow eei(i)$
binary	$v_2, v_1 \leftarrow \mu[p - 1 :]; \mu[p - 1] \leftarrow v_1\ op\ v_2; p \leftarrow p - 1$	sstore	$v, i \leftarrow \mu[p - 1 :];$ $eei(i, v); p \leftarrow p - 2$
jump	$d \leftarrow \mu[p]; p \leftarrow p - 1;$ switch d , table	call	$sz, st, z, y; \mu[p - 3 :];$ $p \leftarrow p - 4; eei(y, z, st, sz)$
jumpi	$z, d \leftarrow \mu[p - 1 :]; p \leftarrow p - 2;$ br $z \neq 0, d$, fall-through	return	$i, s \leftarrow \mu[p - 1 :]; eei(i, s)$
invalid	unreachable	suicide	$id \leftarrow \mu[p]; eei(id)$

A Vulnerabilities

► P1: Arithmetic Overflow (AR). An arithmetic operation may overflow/underflow its maximum or minimum size [27],

which might lead to serious impacts in smart contracts, i.e., transferring invalid amounts of cryptocurrency.

Patch. We apply LLVM sanitizers to guard arithmetic operations. It reverts the execution if any overflow happens.

► P2: Intra-function Reentrancy (iRE). Whenever a smart contract transfers cryptocurrency to another smart contract, the caller will receive a callback from the callee. In iRE attacks, a malicious contract can become a callee and call back into the caller contract to take over its control flow. This may cause the inconsistent invocations of the function to interact in undesirable ways.

Patch. We allocate a global variable as the mutex. In LLVM ASSEMBLY, the patch reverts the execution if the mutex is locked before an external call; otherwise, it locks the mutex to continue the execution. Behind the external call, it unlocks the mutex immediately for security. In the first invocation of the victim function, the mutex is locked. When a malicious contract reenters the victim, the locked mutex raises an exception to revert the exploit transaction.

► P3: Cross-function Reentrancy (cRE). cRE attackers can cause inconsistent states by reentering different functions. **Patch.** To patch it, we need to lock every function with a mutex. EVMBT integrates all smart contract methods into a single LLVM ASSEMBLY function. Therefore, we check and lock the mutex at the *main* function entry and unlock it before every terminator.

► P4: Dangerous tx.origin (OR). *tx.origin* is the original transaction's sender rather than the current sender. A dangerous *tx.origin* may cause an unexpected authorization bypass. **Patch.** If a `getOrigin()` (for lifting *tx.origin*) is detected as vulnerable, we replace it with another EEI function, i.e., `getCaller()` (for lifting *msg.sender*).

► P5: Unchecked Send (US). Smart contracts can continue the execution even if the external calls fail. If an attacker forces the external call to fail, this may cause unexpected behavior in the subsequent program logic [27].

Patch. EVMBT inserts a branch to check the result of external calls (i.e., `CALL`), reverting the transaction if it fails.

References

- [1] 2020. Panoramix. <https://github.com/palkeo/panoramix>.
- [2] 2021. Ethereum Environment Interface. <https://t.ly/VsUvQ>.
- [3] 2021. Etherscan. <https://etherscan.io/>.
- [4] 2021. EVMJIT. <https://github.com/ethereum/evmjit>.
- [5] 2021. Ewasm Contract Interface Specification. <https://t.ly/Uz3po>.
- [6] 2021. ewasm Geth. <https://github.com/ewasm/go-ethereum>.
- [7] 2021. Hera. <https://github.com/ewasm/hera>.
- [8] 2021. Library contracts. <https://jeancvllr.medium.com/solidity-tutorial-all-about-libraries-762e5a3692f9..>
- [9] 2021. LLL. https://lll-docs.readthedocs.io/en/latest/lll_introduction.html.
- [10] 2021. Mythril. <https://github.com/ConsenSys/mythril>.
- [11] 2021. NEAR. <https://docs.near.org/docs/develop/contracts/overview>.
- [12] 2021. Polkadot. <https://wiki.polkadot.network>.
- [13] 2021. RunEVM. <https://github.com/axic/runevm>.
- [14] 2021. Serpent. <https://github.com/ethereum/serpent>.
- [15] 2021. Solana. <https://solana.com/developers>.

- [16] 2021. Solidity. <https://docs.soliditylang.org/>.
- [17] 2021. Vyper. <https://vyper.readthedocs.io>.
- [18] 2021. WebAssembly. <https://webassembly.org/>.
- [19] 2022. Aurora. <https://doc.aurora.dev/>
- [20] 2022. Bytecoder. <https://github.com/mirkosertic/Bytecoder>
- [21] 2022. Ethereum 2.0. <https://github.com/ethereum/consensus-specs/>.
- [22] 2022. Ethereum tests. <https://github.com/ethereum/tests>.
- [23] 2022. JVM Spec. <https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html>
- [24] 2022. JWebAssembly. <https://github.com/i-net-software/JWebAssembly>
- [25] 2022. LLVM WebAssembly backend. <https://llvm.org/>
- [26] 2022. Moonbeam. <https://moonbeam.network/>
- [27] 2024. Smart Contract Weakness. <https://swcregistry.io/docs/>.
- [28] 2024. TeaVM. <https://www.teavm.org/>
- [29] A. Altinay and J. Nash. 2020. BinRec: dynamic binary lifting and recompilation. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys)* (Heraklion, Greece). Article 36, 16 pages. <https://doi.org/10.1145/3342195.3387550>
- [30] H. Chen, M. Pendleton, L. Njilla, and S. Xu. 2020. A Survey on Ethereum Systems Security: Vulnerabilities, Attacks, and Defenses. *ACM Comput. Surv.* 53, 3 (2020). <https://doi.org/10.1145/3391195>
- [31] J. Chen, X. Xia, D. Lo, and J. Grundy. 2021. Maintenance-related concerns for post-deployed Ethereum smart contract development: issues, techniques, and future challenges. *Empir. Softw. Eng.* (2021).
- [32] T. Chen, Z. Li, Y. Zhang, X. Luo, T. Wang, T. Hu, X. Xiao, D. Wang, J. Huang, and X. Zhang. 2019. A Large-Scale Empirical Study on Control Flow Identification of Smart Contracts. In *Proc. of the International Symposium on Empirical Software Engineering and Measurement*. Recife, Brazil, 1–11. <https://doi.org/10.1109/ESEM.2019.8870156>
- [33] V. Chipounov and G. Candea. 2011. Enabling sophisticated analyses of x86 binaries with revgen. In *Proc. of the 41st International Conference on Dependable Systems and Networks Workshops*. Hong Kong SAR.
- [34] DeFillama. 2022. Total Value Locked of DeFi on Ethereum. <https://defillama.com/chain/Ethereum>
- [35] A. Dinaburg and A. Ruef. 2014. Mcsema: Static translation of x86 instructions to llvm. In *ReCon 2014 Conference, Montreal, Canada*.
- [36] T. Durieux, J. Ferreira, R. Abreu, and P. Cruz. 2020. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) (ICSE '20). 530–541. <https://doi.org/10.1145/3377811.3380364>
- [37] J. Ferreira, P. Cruz, T. Durieux, and R. Abreu. 2021. SmartBugs: a framework to analyze solidity smart contracts. In *Proc. of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual, Australia). 1349–1352. <https://doi.org/10.1145/3324884.3415298>
- [38] N. Grech, L. Brent, B. Scholz, and Y. Smaragdakis. 2019. Gigahorse: thorough, declarative decompilation of smart contracts. In *Proc. of the 41st International Conference on Software Engineering* (Montreal, Canada). 1176–1186. <https://doi.org/10.1109/ICSE.2019.00120>
- [39] N. Grech, S. Lagouvardos, I. Tsatiris, and Y. Smaragdakis. 2022. Elipmoc: Advanced Decompilation of Ethereum Smart Contracts. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 77 (apr 2022), 27 pages. <https://doi.org/10.1145/3527321>
- [40] A. Hilbig, D. Lehmann, and M. Pradel. 2021. An Empirical Study of Real-World WebAssembly Binaries: Security, Languages, Use Cases. In *Proceedings of the Web Conference*. Ljubljana, Slovenia, 2696–2708. <https://doi.org/10.1145/3442381.3450138>
- [41] hyperledger labs. 2021. Solidity Compiler for Solana, Substrate, and ewasm. <https://shorturl.at/rBEI5>.
- [42] H. Jin, Z. Wang, M. Wen, W. Dai, Y. Zhu, and D. Zou. 2022. Aroc: An Automatic Repair Framework for On-Chain Smart Contracts. *IEEE Transactions on Software Engineering* 48, 11 (2022), 4611–4629. <https://doi.org/10.1109/TSE.2021.3123170>
- [43] L. Jin, Y. Cao, Y. Chen, D. Zhang, and S. Campanoni. 2023. EXGEN: Cross-platform, Automated Exploit Generation for Smart Contract Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 20, 1 (2023), 650–664. <https://doi.org/10.1109/TDSC.2022.3141396>
- [44] S. Kalra, S. Goel, M. Dhawan, and S. Sharma. 2018. ZEUS: Analyzing Safety of Smart Contracts. In *25th Annual Network and Distributed System Security Symposium, NDSS*. San Diego, USA.
- [45] Donald E Knuth. 2014. *Art of computer programming, volume 2: Seminumerical algorithms*. Addison-Wesley Professional.
- [46] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis transformation. In *International Symposium on Code Generation and Optimization (CGO)*. California, USA, 75. <https://doi.org/10.5555/977395.977673>
- [47] D. Lehmann, J. Kinder, and M. Pradel. 2020. Everything Old is New Again: Binary Security of WebAssembly. In *Proc. of the 29th USENIX Security Symposium*. USA, Article 13, 18 pages. <https://doi.org/10.5555/3489212.3489225>
- [48] Z. Liu, Y. Yuan, S. Wang, and Y. Bao. 2022. SoK: Demystifying Binary Lifters Through the Lens of Downstream Applications. In *Proc. of the 2022 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA, 1100–1119. <https://doi.org/10.1109/SP46214.2022.9833799>
- [49] L. Luu, D. Chu, H. Olickel, P. Saxena, and A. Hobor. 2016. Making Smart Contracts Smarter. In *Proc. of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. Vienna, Austria.
- [50] G. Karame M. Rodler, W. Li and L. Davi. 2021. EVMPatch: Timely and Automated Patching of Ethereum Smart Contracts. In *30th USENIX Security Symposium (USENIX Security 21)*. Virtual, 1289–1306.
- [51] S. Malwa. 2022. 10M Ether Now Locked on Eth 2.0 Staking Contract. <https://www.coindesk.com/tech/2022/03/11/10m-ether-now-locked-on-eth-20-staking-contract/>.
- [52] Microsoft. 2021. mctoll. <https://github.com/microsoft/llvm-mctoll>.
- [53] T. Nguyen, L. Pham, and J. Sun. 2021. SGUARD: Towards Fixing Vulnerable Smart Contracts Automatically. In *Proc. of the IEEE Symposium on Security and Privacy*. Virtual.
- [54] Trail of Bits. 2021. Remill. <https://github.com/lifting-bits/remill>.
- [55] D. Park, Y. Zhang, and G. Rosu. 2020. End-to-End Formal Verification of Ethereum 2.0 Deposit Smart Contract. In *Computer Aided Verification*. Springer International Publishing, Cham.
- [56] J. Ratcliff and D. Metzener. 1988. Pattern Matching: The Gestalt Approach. In *Dr. Dobbs's Journal*.
- [57] Second-State. 2021. SOLL. <https://github.com/second-state/SOLL>.
- [58] M. Šipek, M. Žagar, N. Drašković, and B. Mihaljević. 2022. Blockchain as an IoT Intermediary. In *New Realities, Mobile Systems and Applications*, Michael E. Auer and Thrasylvoulos Tsiatsos (Eds.). Springer International Publishing, Cham.
- [59] H. Cai H. Wang W. Chen, X. Luo. 2024. Towards Smart Contract Fuzzing on GPU. In *Proc. of the 2024 IEEE Symposium on Security and Privacy (SP)*. San Francisco, USA.
- [60] G. Wood. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151, 2014 (2014), 1–32.
- [61] X. Yu, O. Al-Bataineh, D. Lo, and A. Roychoudhury. 2020. Smart Contract Repair. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 4, Article 27 (2020), 32 pages. <https://doi.org/10.1145/3402450>
- [62] Y. Zhang, S. Ma, J. Li, K. Li, S. Nepal, and D. Gu. 2020. SMARTSHIELD: Automatic Smart Contract Protection Made Easy. In *Proc. of the 27th International Conference on Software Analysis, Evolution and Reengineering, SANER*. London, Britain, 23–34. <https://doi.org/10.1109/SANER48275.2020.9054825>
- [63] S. Zheng, H. Wang, L. Wu, G. Huang, and X. Liu. 2024. VM Matters: A Comparison of WASM VMs and EVMs in the Performance of Blockchain Smart Contracts. 9, 2, Article 5 (Mar 2024), 24 pages. <https://doi.org/10.1145/3641103>

Received 2024-02-29; accepted 2024-04-01